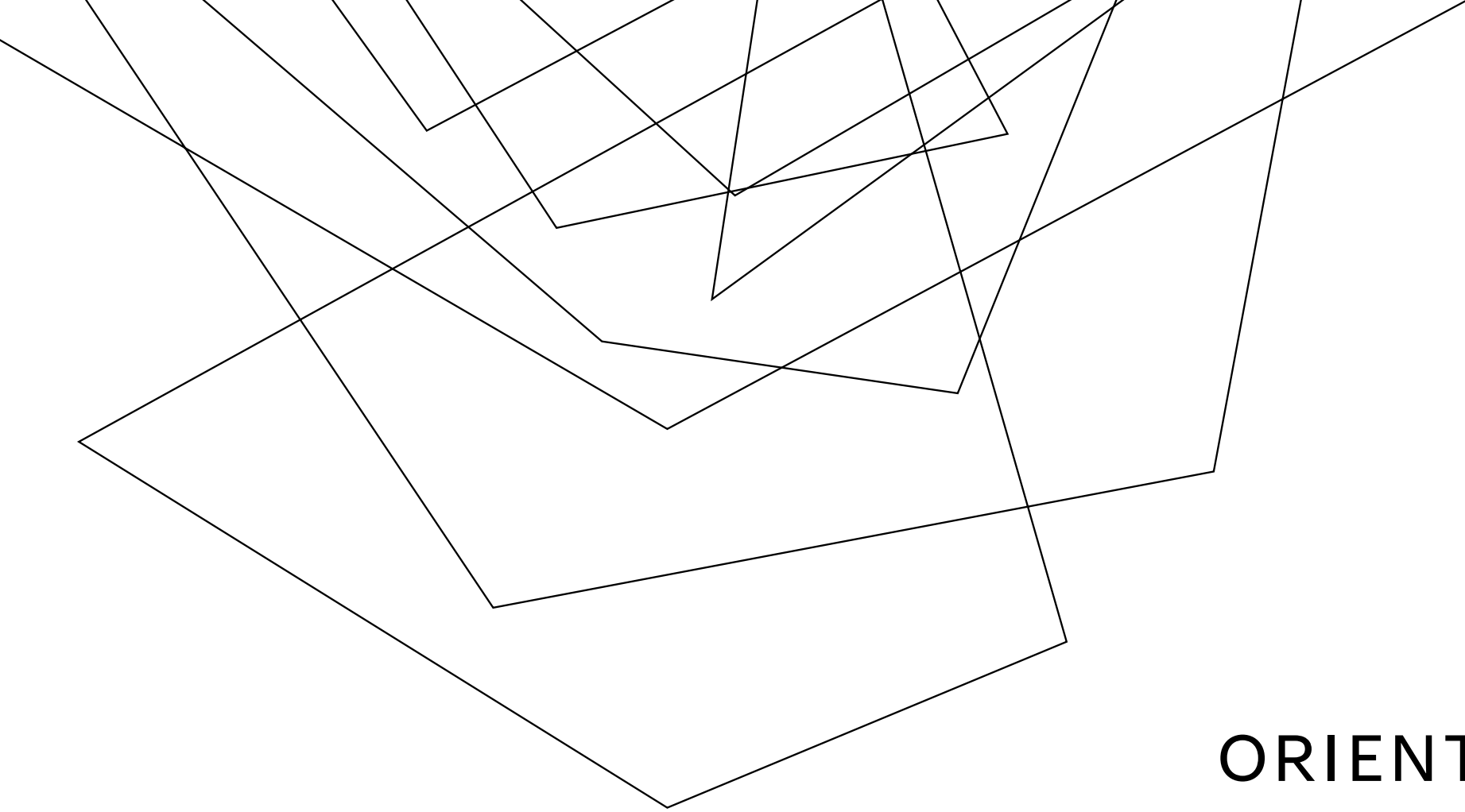




ORIENTAÇÃO A OBJETOS

TALITA VIEIRA RIBEIRO



PILARES DA ORIENTAÇÃO A OBJETOS

Herança

RELACIONAMENTOS

- Os objetos interagem entre si através de **relacionamentos**, que são mecanismos pelos quais um objeto acessa recursos providos por outro.
- São tipos possíveis de relacionamentos entre classes:
 - Dependência
 - Associação
 - Agregação (todo-parte + fraca)
 - A parte existe sem o todo
 - O todo NÃO controla o ciclo de vida da parte
 - Composição (todo-parte + forte)
 - A parte NÃO existe sem o todo
 - O todo CONTROLA o ciclo de vida da parte
 - **Generalização (Herança)**
 - Implementação (Interface)

HERANÇA

- Note as duas classes a seguir:

```
public class Carro {  
    private String placa;  
    private String marca;  
    private String modelo;  
    private int ano;  
    private int quilometragem;  
    private float valorBaseVenda;  
    private int numeroPortas;  
    private String tipoCarroceria;  
    private int capacidadeBagagem;  
  
    public void exibirInformacoes() {...}  
    public void realizarRevisao(int novoKm) {...}  
    public float calcularValorFinal() {...}  
}
```

```
public class Caminhao {  
    private String placa;  
    private String marca;  
    private String modelo;  
    private int ano;  
    private int quilometragem;  
    private float valorBaseVenda;  
    private int numeroEixos;  
    private int capacidadeCarga;  
  
    public void exibirInformacoes() {...}  
    public void realizarRevisao(int novoKm) {...}  
    public float calcularValorFinal() {...}  
}
```

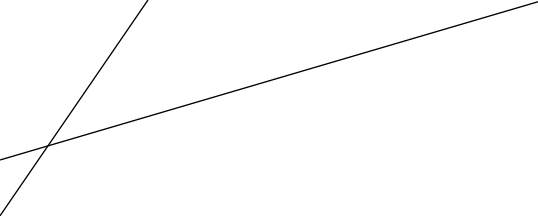
Definindo dessa forma, teríamos que representar os atributos e métodos comuns nas duas classes (duplicidade de código!)

HERANÇA

- Mecanismo pelo qual classes herdam atributos e métodos através de um relacionamento hierárquico.
- É uma forma de dizer que uma classe "é um tipo de" outra classe.

HERANÇA

- Exemplos:
 - Um professor **é um tipo de** funcionário.
 - Um pentágono **é um tipo de** polígono.
 - Um carro **é um tipo de** veículo.



HERANÇA

Na herança, temos sempre uma classe definida de forma genérica que, posteriormente, é refinada em classes mais específicas.

HERANÇA

- Exemplos:
 - Um **professor** é um tipo de **funcionário**.
 - Um **pentágono** é um tipo de **polígono**.
 - Um **carro** é um tipo de **veículo**.

Classes específicas
ou subclasses

Classes genéricas
ou superclasses

HERANÇA

- Repare que podemos ter várias classes específicas para uma mesma classe genérica:
 - Um **professor** é um tipo de **funcionário**
 - Uma **secretária** é um tipo de **funcionário**
 - Um **diretor** é um tipo de **funcionário**

 - Um **retângulo** é um tipo de **polígono**
 - Um **pentágono** é um tipo de **polígono**
 - Um **hexágono** é um tipo de **polígono**

 - Uma **moto** é um tipo de **veículo**
 - Um **carro** é um tipo de **veículo**
 - Um **caminhão** é um tipo de **veículo**

HERANÇA

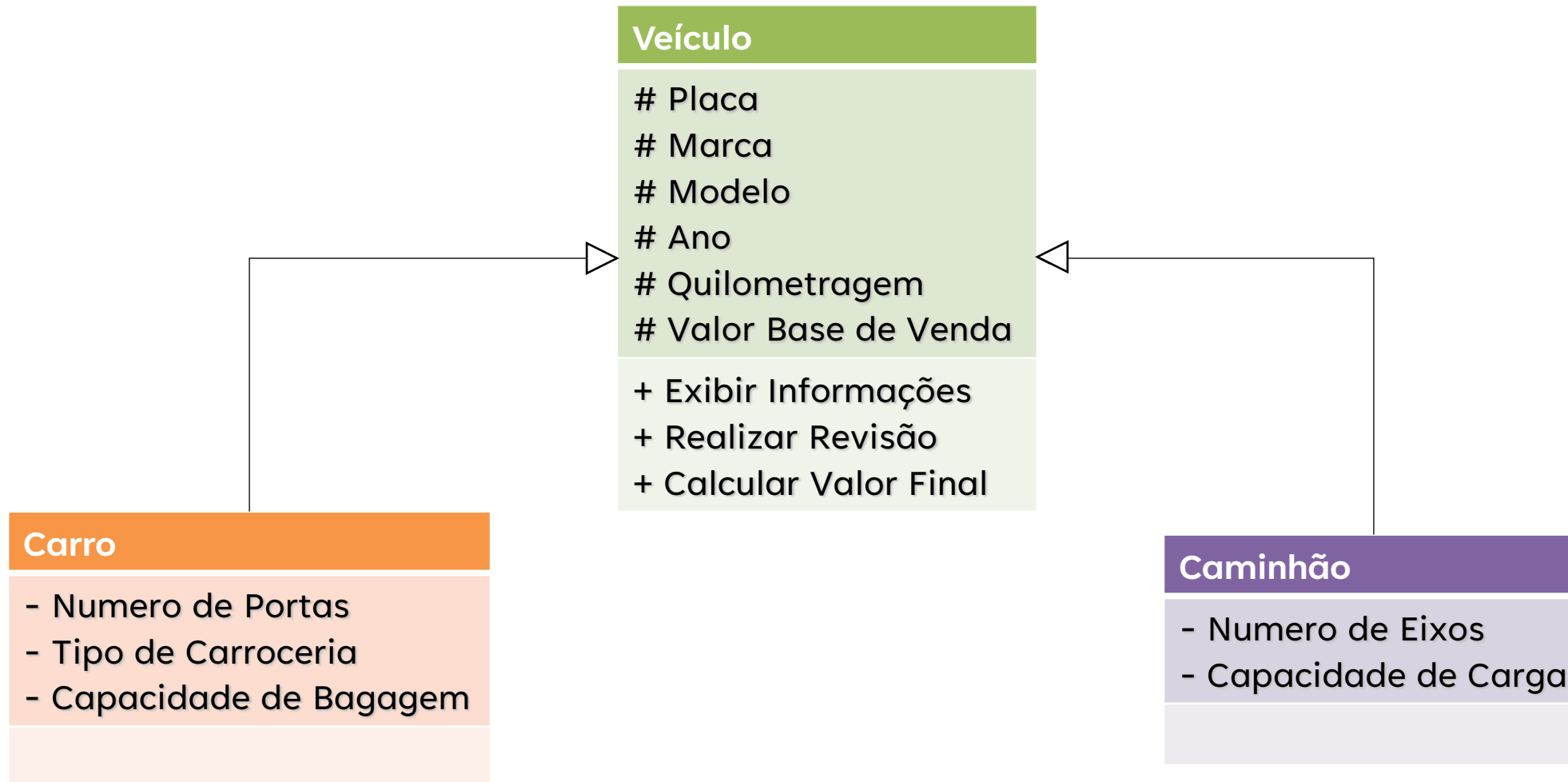
- Conforme dito anteriormente, a classe genérica compartilha atributos e métodos com as classes específicas.
- Ou seja, as classes específicas herdam atributos e métodos **public/protected/package*** da classe genérica.
- Além disso, as classes específicas podem ter atributos e métodos somente seus.

HERANÇA

- No exemplo:
 - Um **carro** é um tipo de **veículo**
 - Um **caminhão** é um tipo de **veículo**
- A classe **Veiculo** vai conter os atributos e métodos comuns a todos os veículos.
- As classes **Carro** e **Caminhão** vão conter os atributos e métodos específicos de carros e caminhões, respectivamente.

Legenda:
- visibilidade private
visibilidade protected
+ visibilidade public

HERANÇA



HERANÇA

- Herança Simples x Herança Múltipla
 - Se uma classe herda de apenas uma superclasse, temos uma herança simples.
 - Se uma classe herda de diversas superclasses, temos uma herança múltipla.
- **Exemplo:** Carro anfíbio é um tipo de carro e barco, ao mesmo tempo. O carro anfíbio possui características de carro e de barco.



EM JAVA

HERANÇA EM JAVA

- A linguagem Java não possui herança múltipla, somente herança simples.
- Para definir a herança entre duas classes devemos usar a palavra reservada `extends` na definição da subclasse.

```
public class <nome da classe> extends <nome da superclasse>
```

- Em Java, a subclasse herda todos os atributos e métodos definidos como **public** ou **protected** na superclasse.
- Os atributos e métodos definidos como **private** **NÃO são herdados** pelas subclasses e os **package (default)** só serão herdados se as classes estiverem no mesmo pacote.

HERANÇA EM JAVA

```
public class Veiculo {  
    protected String placa;  
    protected String marca;  
    protected String modelo;  
    protected int ano;  
    protected int quilometragem;  
    protected float valorBaseVenda;
```

Atributos **protected** têm a mesma visibilidade de atributos **package** com o adicional da visibilidade dos seguintes casos relacionados à herança:

- **Superclasse** só acessa atributos **protected** de suas **subclasses** se estes forem os definidos na própria superclasse;
- **Subclasses** só acessam atributos **protected** de sua **superclasse** se estes forem os herdados da superclasse.

```
    public Veiculo(String placa, String marca, String modelo, int ano, int quilometragem, float valorBaseVenda) {  
        this.placa = placa;  
        this.marca = marca;  
        this.modelo = modelo;  
        this.ano = ano;  
        this.kilometragem = quilometragem;  
        this.valorBaseVenda = valorBaseVenda;  
    }  
}
```


HERANÇA EM JAVA

```
public class Carro extends Veiculo {  
    private int numeroPortas;  
    private String tipoCarroceria;  
    private int capacidadeBagagem;  
}
```

Note que a classe **Carro** possui os atributos:

- placa
- marca
- modelo
- ano
- quilometragem
- valorBase
- numeroPortas
- tipoCarroceria
- capacidadeBagagem

HERANÇA EM JAVA

```
public class Carro extends Veiculo {  
    private int numeroPortas;  
    private String tipoCarroceria;  
    private int capacidadeBagagem;  
  
    public Carro(String placa, String marca, String modelo, int ano, int quilometragem, float  
        valorBaseVenda, int numeroPortas, String tipoCarroceria, int capacidadeBagagem) {  
        super(placa, marca, modelo, ano, quilometragem, valorBaseVenda);  
        this.numeroPortas = numeroPortas;  
        this.tipoCarroceria = tipoCarroceria;  
        this.capacidadeBagagem = capacidadeBagagem;  
    }  
}
```

Usamos a notação **super()** para executar o construtor da superclasse.

Em seguida, os atributos da classe **Carro** são inicializados.

HERANÇA EM JAVA

```
public class Caminhao extends Veiculo {  
    private int numeroEixos;  
    private int capacidadeCarga;  
}
```

Note que a classe **Caminhão** possui os atributos:

- placa
- marca
- modelo
- ano
- quilometragem
- valorBase
- numeroEixos
- capacidadeCarga

HERANÇA EM JAVA

```
public class Caminhao extends Veiculo {  
    private int numeroEixos;  
    private int capacidadeCarga;
```

```
    public Caminhao(String placa, String marca, String modelo, int ano, int quilometragem, float  
        valorBaseVenda, int numeroEixos, int capacidadeCarga) {  
        super(placa, marca, modelo, ano, quilometragem, valorBaseVenda);  
        this.numeroEixos = numeroEixos;  
        this.capacidadeCarga = capacidadeCarga;  
    }  
}
```

Usamos a notação **super()** para executar o construtor da superclasse.

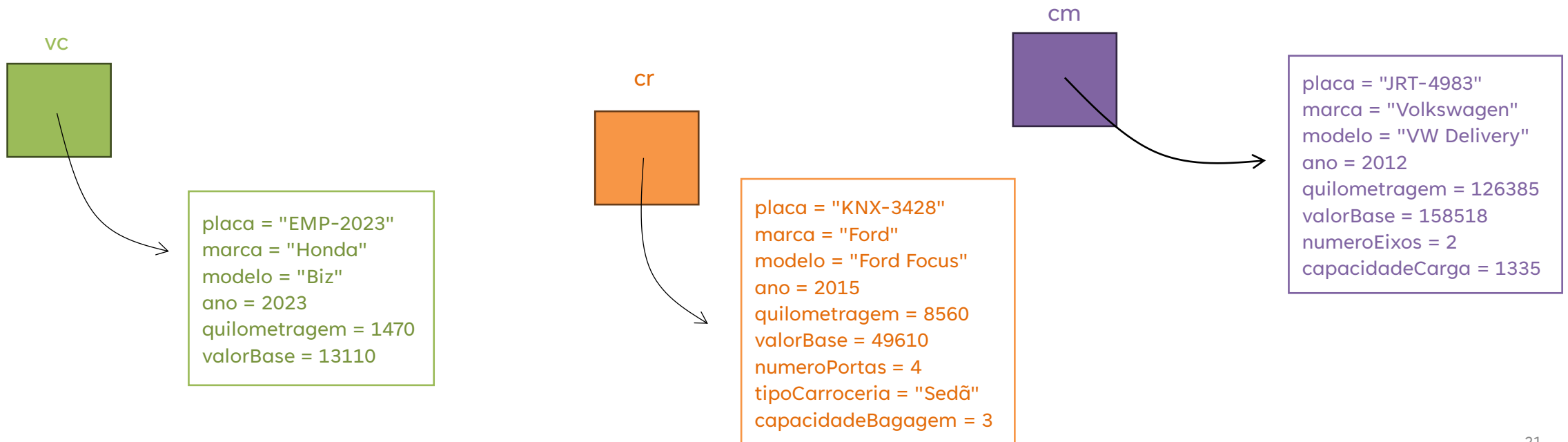
Em seguida, os atributos da classe **Caminhão** são inicializados.

HERANÇA EM JAVA

```
Veículo vc = new Veiculo("EMP-2023", "Honda", "Biz", 2023, 1470, 13110);
```

```
Carro cr = new Carro("KNX-3428", "Ford", "Ford Focus", 2015, 8560, 49610, 4, "Sedã", 3);
```

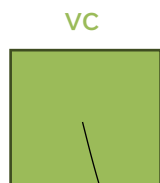
```
Caminhao cm = new Caminhao("JRT-4983", "Volkswagen", "VW Delivery", 2012, 126385, 158518, 2, 1335);
```



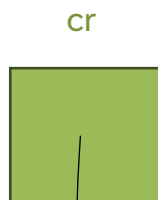
A diferença aqui é que somente métodos declarados em **Veículo** é que poderão ser chamados a partir dos objetos cr e cm.

HERANÇA EM JAVA

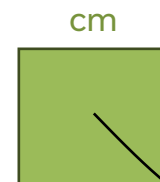
```
Veículo vc = new Veiculo("EMP-2023", "Honda", "Biz", 2023, 1470, 13110);  
Veículo cr = new Carro("KNX-3428", "Ford", "Ford Focus", 2015, 8560, 49610, 4, "Sedã", 3);  
Veículo cm = new Caminhao("JRT-4983", "Volkswagen", "VW Delivery", 2012, 126385, 158518, 2,  
1335);
```



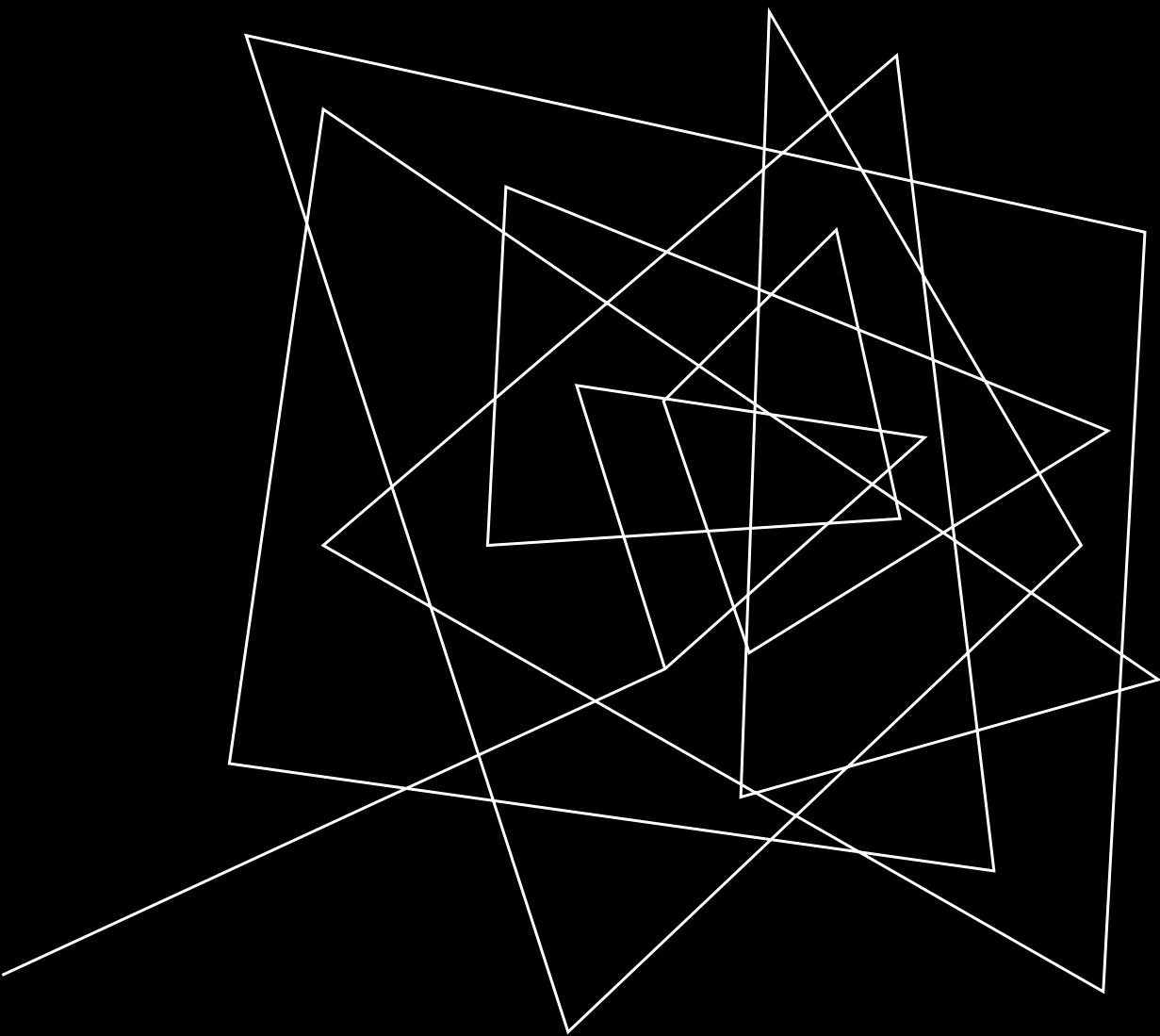
placa = "EMP-2023"
marca = "Honda"
modelo = "Biz"
ano = 2023
quilometragem = 1470
valorBase = 13110



placa = "KNX-3428"
marca = "Ford"
modelo = "Ford Focus"
ano = 2015
quilometragem = 8560
valorBase = 49610
numeroPortas = 4
tipoCarroceria = "Sedã"
capacidadeBagagem = 3



placa = "JRT-4983"
marca = "Volkswagen"
modelo = "VW Delivery"
ano = 2012
quilometragem = 126385
valorBase = 158518
numeroEixos = 2
capacidadeCarga = 1335

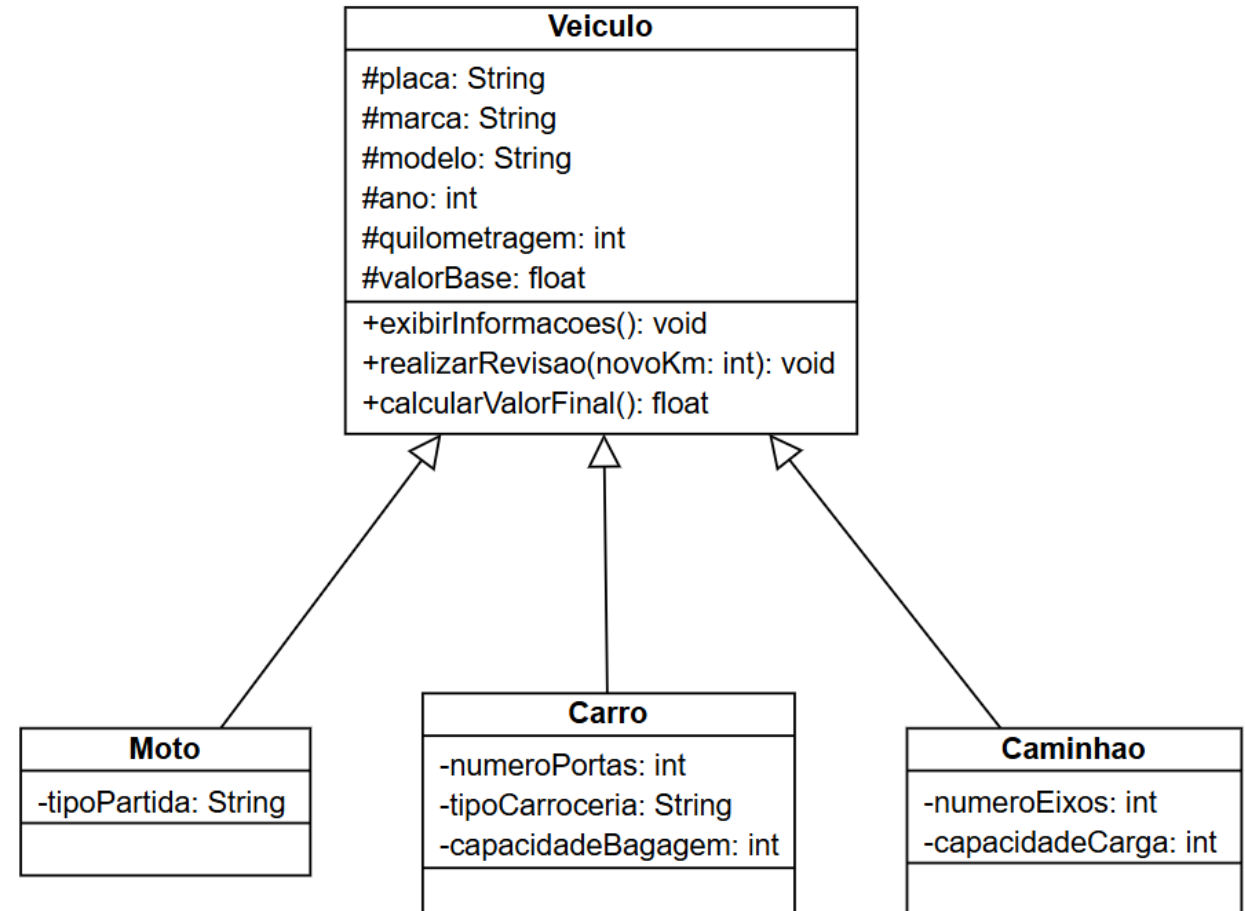


PRATICANDO

PRATICANDO

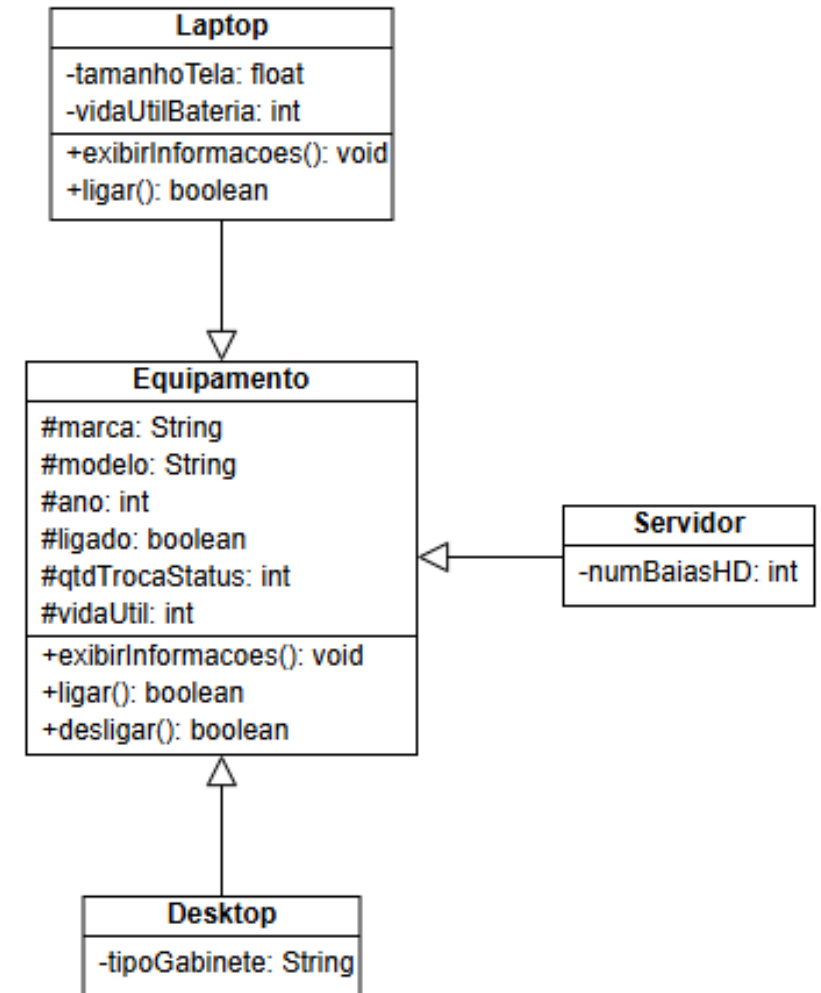
1. Em um novo projeto (Concessionaria), implemente o seguinte diagrama de classes, considerando as seguintes informações para implementação dos métodos de Veículo:

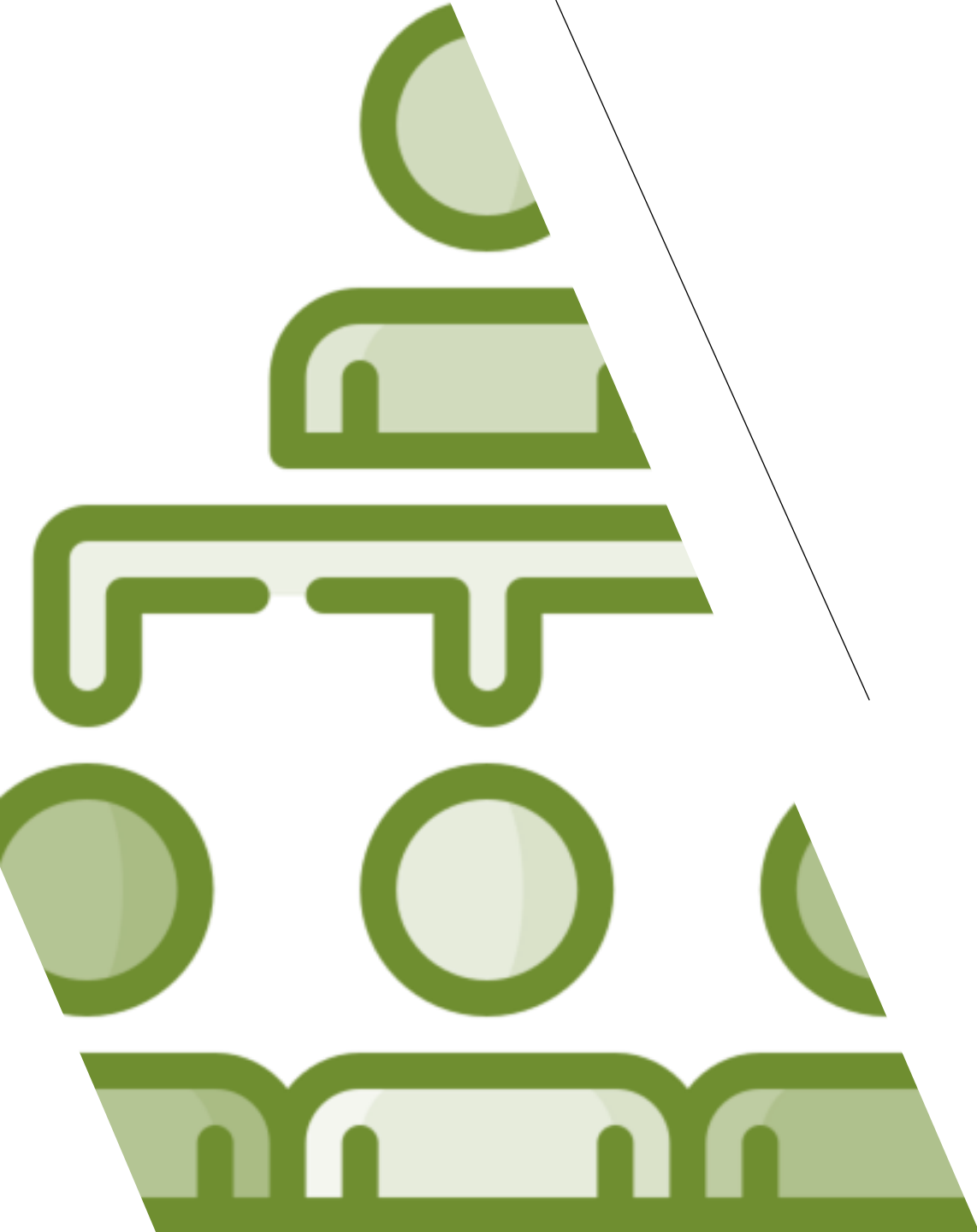
- `exibirInformacoes()`: exibe as informações dos atributos do veículo (somente as presentes na superclasse).
- `realizarRevisao(int novoKM)`: atualiza a quantidade de quilometragem, somente se for maior que a quilometragem já registrada pro Veículo.
- `calcularValorFinal()`: calcula o valor final de venda com base no valor base, no ano e na quilometragem do veículo:
 - Veículos com quilometragem **abaixo** de 15km por ano de idade: o valor de venda será o valor base com desconto de **0,5%** sobre o valor base a cada 1 ano de idade do veículo.
 - Veículos com quilometragem **acima** de 15km por ano de idade: o valor de venda será o valor base com desconto de **1%** sobre o valor base a cada 1 ano de idade do veículo.



PRATICANDO

2. Em um novo projeto (GerenciamentoEletrônicos), implemente o seguinte diagrama de classes, considerando as seguintes informações para implementação dos métodos de cada classe:
- Equipamento:
 - `exibirInformacoes()`: exibe as informações dos atributos do equipamento (somente as presentes na superclasse).
 - `ligar()`: se já estiver ligado ou se `vidaUtil < 1`, retorna `false` e não altera nada. Caso contrário, define `ligado = true`, incrementa `qtdTrocaStatus` em 1, decrementa `vidaUtil` em 1 e retorna `true`.
 - `desligar()`: se já estiver desligado, retorna `false` e não altera nada. Caso contrário, define `ligado = false`, incrementa `qtdTrocaStatus` em 1 e retorna `true`.
 - Laptop:
 - `exibirInformacoes()`: chama o `exibirInformacoes()` da superclasse e adiciona as informações dos atributos de laptop.
 - `ligar()`: se já estiver ligado ou se `vidaUtil < 1` ou se `vidaUtilBateria < 1`, retorna `false` e não altera nada. Caso contrário, define `ligado = true`, incrementa `qtdTrocaStatus` em 1, decrementa `vidaUtil` em 1, decrementa `vidaUtilBateria` em 1 e retorna `true`.





CLASSES ABSTRATAS

**Superclasses que não Instanciam
Objetos**

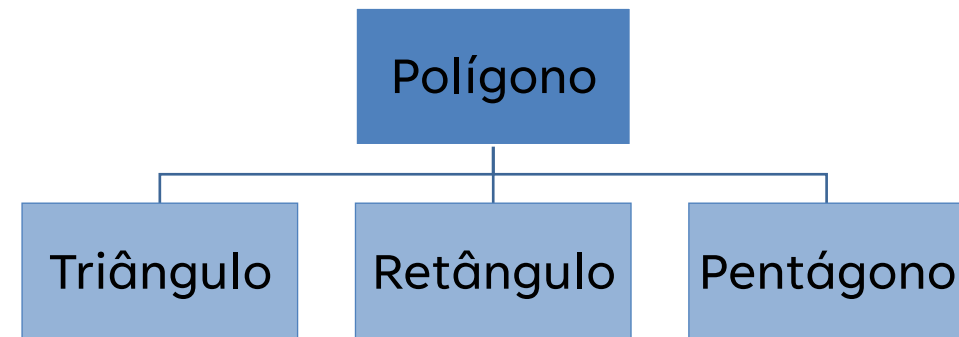
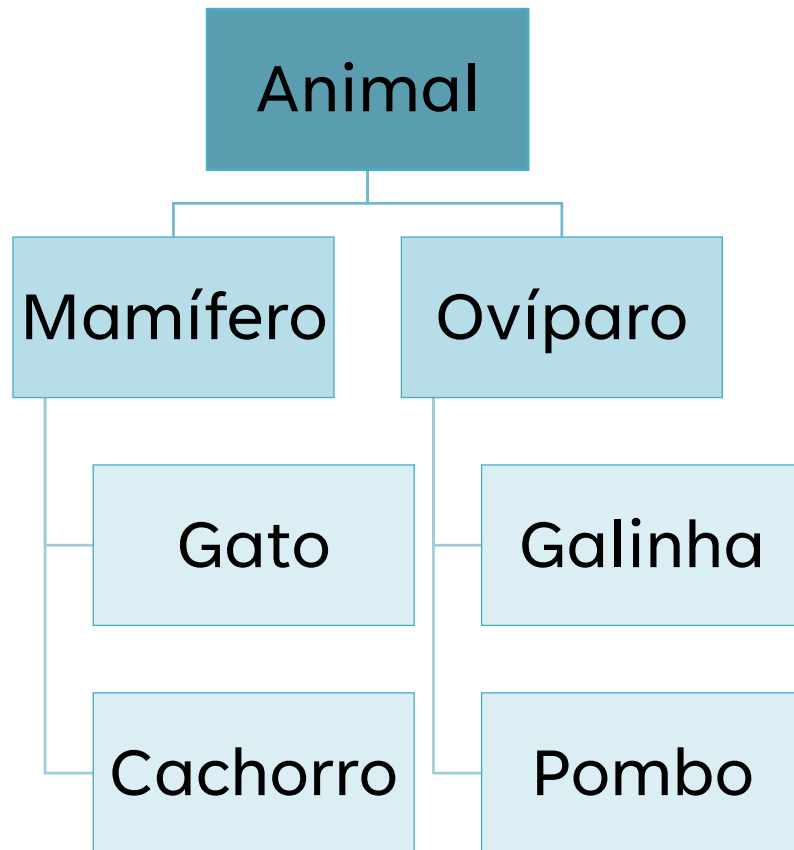
CLASSES ABSTRATAS

- No exemplo da concessionária temos:
 - **Moto** é um tipo de **Veículo**
 - **Carro** é um tipo de **Veículo**
 - **Caminhão** é um tipo de **Veículo**
 - **Veículo** possui placa, marca, modelo, ano e valor base de venda
- Perguntas:
 - Se algum cliente entrar na concessionária e pedir para comprar um **Veículo**, o vendedor saberá o que deve vender?
 - Se a concessionária anunciar que está vendendo um **Veículo**, o cliente tem condições de saber exatamente o que está comprando?
- Isso ocorre porque **Veículo** é uma abstração para **Moto**, **Carro** e **Caminhão**.

CLASSES ABSTRATAS

- **Classes Abstratas** são classes que representam abstrações (conceitos) e não objetos concretos do mundo que estamos representando.
- São usadas como **moldes para a criação** de outras classes e podem conter atributos e comportamentos.
- No exemplo da concessionária Veículo é uma classe abstrata. Nos últimos Dojos também estávamos diante de classes abstratas.
- **Veículo** define um molde a partir do qual as classes concretas **Moto**, **Carro** e **Caminhão** são definidas.

CLASSES ABSTRATAS





EM JAVA

CLASSES ABSTRATAS

- Classes abstratas são definidas **em Java** com o uso da palavra `abstract` na sua definição.
- Por não representarem elementos concretos, não é possível criar objetos a partir de classes abstratas.

```
public abstract class Animal {  
    protected String nome;  
    protected int anoNascimento;  
  
    public Animal(String nome, int anoNascimento){  
        this.nome = nome;  
        this.anoNascimento = anoNascimento;  
    }  
  
    public void emitirSom() {  
        System.out.println("???");  
    }  
}  
...  
Animal a = new Animal("Bob", 2011);
```

Erro de compilação

CLASSES ABSTRATAS

- Classes abstratas são definidas **em Java** com o uso da palavra `abstract` na sua definição.
- Por não representarem elementos concretos, não é possível criar objetos a partir de classes abstratas.

```
public abstract class Animal {  
    protected String nome;  
    protected int anoNascimento;  
  
    public Animal(String nome, int anoNascimento){  
        this.nome = nome;  
        this.anoNascimento = anoNascimento;  
    }  
  
    public void emitirSom() {  
        System.out.println("???");  
    }  
}  
...  
Animal a;
```

É possível declarar um objeto como sendo do tipo de uma classe abstrata.

CLASSES ABSTRATAS

- Classes abstratas são definidas **em Java** com o uso da palavra `abstract` na sua definição.
- Por não representarem elementos concretos, não é possível criar objetos a partir de classes abstratas.

```
public abstract class Animal {  
    protected String nome;  
    protected int anoNascimento;  
  
    public Animal(String nome, int anoNascimento){  
        this.nome = nome;  
        this.anoNascimento = anoNascimento;  
    }  
  
    public void emitirSom() {  
        System.out.println("???");  
    }  
}
```

Faz sentido um comportamento genérico de emitir som “???” para todo e qualquer animal?

MÉTODOS ABSTRATOS

- Quando definimos uma classe abstrata, muitas vezes nos deparamos com métodos que fazem sentido para aquela classe, mas não temos como implementá-lo, porque a classe abstrata representa apenas uma abstração.

Embora os demais métodos possam ter implementação genérica com os dados de Polígono, a verificação de validade de um polígono específico varia entre os tipos concretos. Por isso, `ehValido()` **não deve ser implementado** nesta classe base.

```
public abstract class Poligono {
    protected int quantidadeVertices;
    protected Ponto[] vertices;
    protected SegmentoReta[] lados;

    public String determinarTipo() {
        //identificar a partir da quantidade de vértices.
    }

    public float perimetro() {
        //somar o tamanho de todos os lados.
    }

    public float area() {
        //usar a fórmula de Shoelace que só precisa dos vértices para
        calcular a área.
    }

    public boolean ehValido() {
        //cada polígono tem uma forma de validação específica de sua
        estrutura, então não tem como fazer uma implementação geral!
    }
}
```

MÉTODOS ABSTRATOS

- Métodos em que sua implementação só faz sentido nas subclasses concretas, são chamados de **métodos abstratos**.

```
public abstract class Animal {  
    protected String nome;  
    protected int anoNascimento;  
  
    public Animal(String nome, int anoNascimento){  
        this.nome = nome;  
        this.anoNascimento = anoNascimento;  
    }  
  
    public abstract void emitirSom();  
}
```

```
public abstract class Poligono {  
    protected int quantidadeVertices;  
    protected Ponto[] vertices;  
    protected SegmentoReta[] lados;  
  
    public abstract boolean ehValido();  
}
```

Métodos abstratos não
possuem implementação e
por isso se encerram no ;

MÉTODOS ABSTRATOS

- As classes **concretas** que herdam da classe abstrata devem, obrigatoriamente , implementar os métodos abstratos.

```
public class Triangulo extends Poligono {  
  
    public boolean ehValido() {  
        return  
            lados[0].tamanho() > lados[1].tamanho() + lados[2].tamanho() &&  
            lados[1].tamanho() > lados[2].tamanho() + lados[0].tamanho() &&  
            lados[2].tamanho() > lados[0].tamanho() + lados[1].tamanho();  
    }  
}
```

As classes concretas que herdam de Poligono devem implementar o método ehValido(). Repara que na assinatura do método da classe concreta a palavra “abstract” é removida.

MÉTODOS ABSTRATOS

- As classes **abstratas** que herdam de outra classe abstrata não tem qualquer obrigação de implementar os métodos abstratos de sua superclasse, mas se quiserem, podem.

```
public abstract class Animal {  
    protected String nome;  
    protected int anoNascimento;  
  
    public Animal(String nome, int anoNascimento){  
        this.nome = nome;  
        this.anoNascimento = anoNascimento;  
    }  
  
    public abstract void emitirSom();  
}
```

```
public abstract class Mamifero extends Animal {  
  
}
```

OU

```
public abstract class Mamifero extends Animal {  
  
    public void emitirSom() {  
        System.out.println("Tenho pelos em meu  
corpo");  
    }  
  
}
```

MODIFICADOR **FINAL** EM DEFINIÇÃO DE CLASSES

- Para evitar que sejam criadas subclasses a partir de uma classe podemos defini-la com o modificador final.
- Exemplo:

```
public abstract class Mamifero extends Animal{  
    public void emitirSom() {  
        System.out.println("Tenho pelos em  
meu corpo");  
    }  
}  
  
public final class Gato extends Mamifero {  
    public void emitirSom() {  
        System.out.println("miau");  
    }  
}  
  
public class Siames extends Gato {  
}
```

Se tentarmos criar uma subclasse de **Gato** haverá um erro de compilação, porque Gato está definido como **final**.

Conceitualmente queremos dizer que tudo de Gato deverá ser tratado na respectiva classe, não devendo serem criadas classes mais específicas que Gato.

MODIFICADOR **FINAL** EM DEFINIÇÃO DE MÉTODOS

- Um método da superclasse poderá ser **sobrescrito** na subclasse.

Sobrecarga (Overloading)

```
public abstract class Animal {  
    public abstract void emitirSom();  
  
    public void emitirSom(String som){  
        System.out.println(som);  
    }  
}
```

```
public abstract class Mamifero extends Animal{  
    public void emitirSom() {  
        System.out.println("Tenho pelos em meu  
corpo"); }  
}
```

Sobrescrita (Overriding)

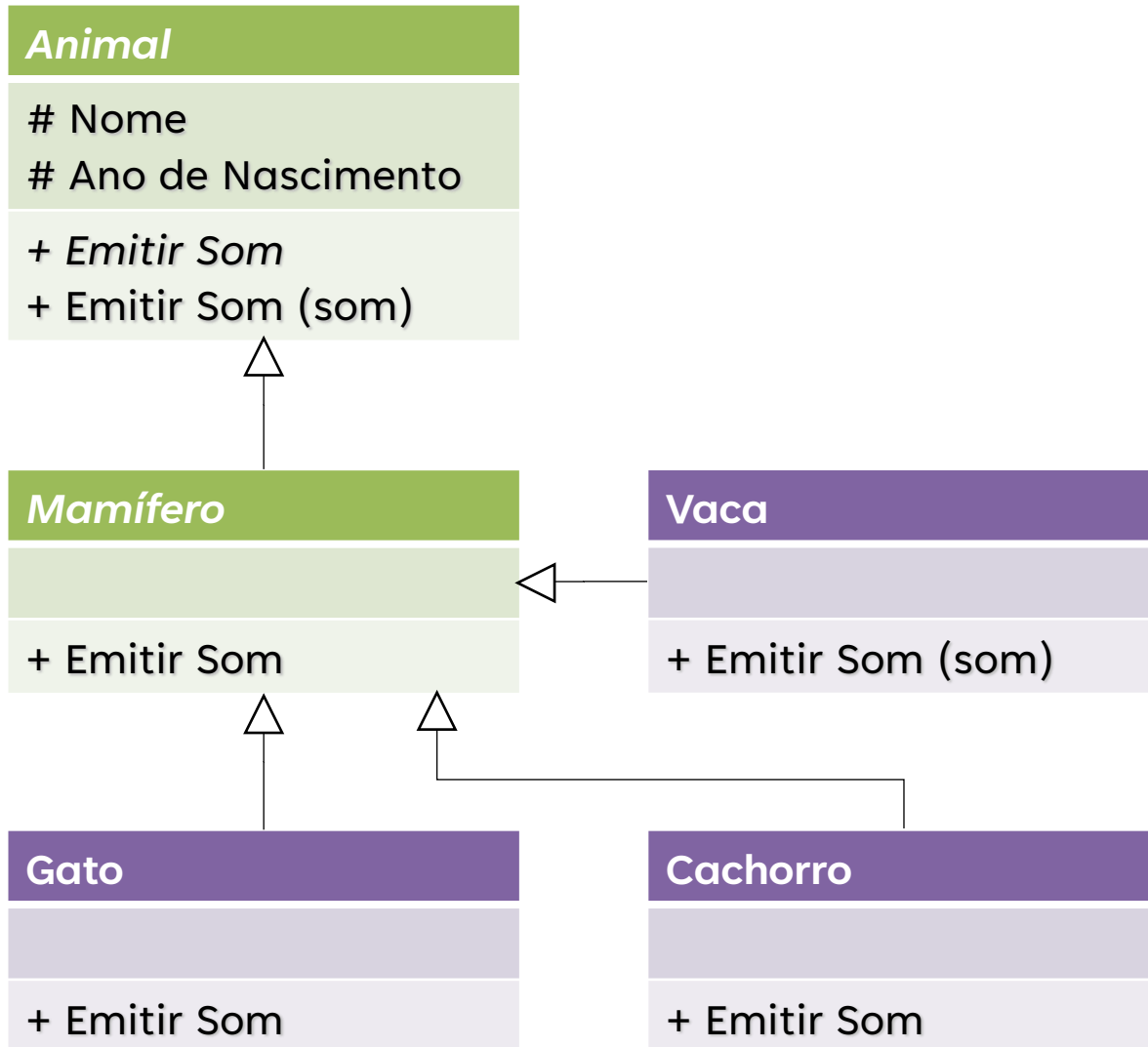
```
public final class Gato extends Mamifero {  
    public void emitirSom() {  
        System.out.println("Miau"); }  
}
```

```
public class Cachorro extends Mamifero {  
    public void emitirSom() {  
        System.out.println("Auau"); }  
}
```

```
public final class Vaca extends Mamifero {  
    public void emitirSom(String som) {  
        System.out.print("Queria fazer muuu, mas vou fazer ");  
        super.emitirSom(som); }  
}
```

Observe que é possível realizar a chamada de um método da superclasse através da palavra super seguida do nome do método: super.nomeDoMetodo();

SOBRESCRITA



```
Cachorro cachorrinho = new
Cachorro("André", "20/08/2020");
cachorrinho.emitirSom(); //chama o emitir
som da classe Cachorro.
cachorrinho.emitirSom("grr"); //chama o
emitir som da classe Animal.
```

```
Vaca mimosa = new Vaca("Vivi",
"01/04/2018");
mimosa.emitirSom(); //chama o emitir som da
classe Mamífero.
mimosa.emitirSom("béee"); //chama o emitir
som da classe Vaca.
```


MODIFICADOR **FINAL** EM DEFINIÇÃO DE MÉTODOS

- Podemos usar o modificador final para evitar que um método seja sobrescrito pelas subclasses.
- Exemplo:

```
public abstract class Mamifero extends Animal {  
    public void emitirSom() {  
        System.out.println("Tenho pelos em meu corpo"); }  
  
    public final boolean bebeLeite() {  
        return true; }  
}  
  
public class Cachorro extends Mamifero {  
    public void emitirSom() {  
        System.out.println("Auau"); }  
  
    public boolean bebeLeite() {  
        return false; }  
}
```

Se tentarmos sobrescrever o método **bebeLeite()**, haverá um erro de compilação, pois ele está definido, na superclasse, como final.

A ideia aqui é garantir que se existe um comportamento final na superclasse, todas as suas subclasses se comportam exatamente igual ao descrito no método final, não tendo como ser diferente.

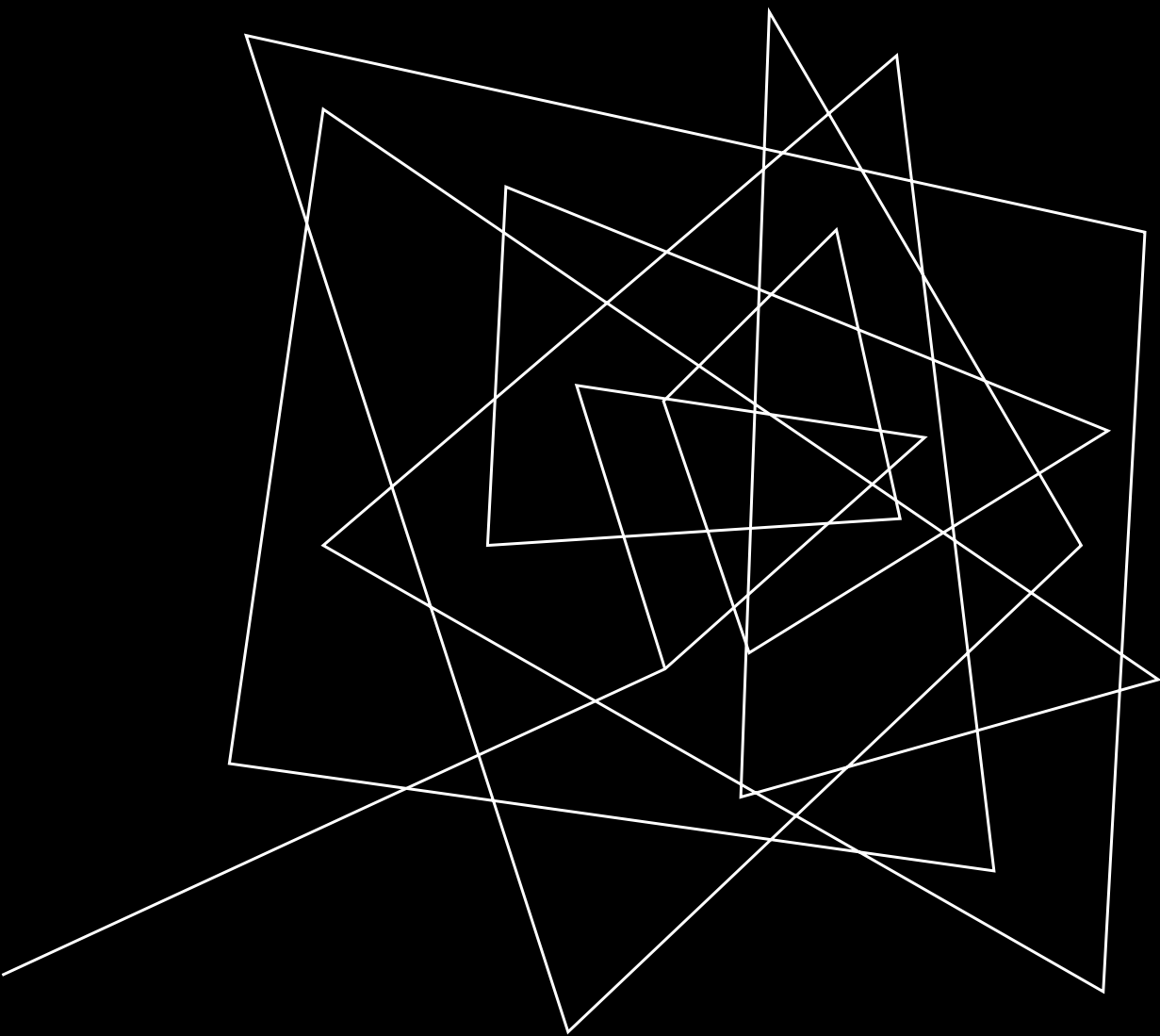
MODIFICADOR **FINAL** EM DEFINIÇÃO DE MÉTODOS

- Podemos usar o modificador final para evitar que um método seja sobrescrito pelas subclasses.

- Exemplo:

```
public abstract class Poligono {  
    protected int quantidadeVertices;  
    protected Ponto[] vertices;  
    protected SegmentoReta[] lados;  
  
    public String determinarTipo() {  
        //identificar a partir da quantidade de vértices.  
    }  
  
    public final float perimetro() {  
        //somar o tamanho de todos os lados.  
    }  
  
    public float area() {  
        //usar a fórmula de Shoelace que só precisa dos vértices para calcular a área.  
    }  
}
```

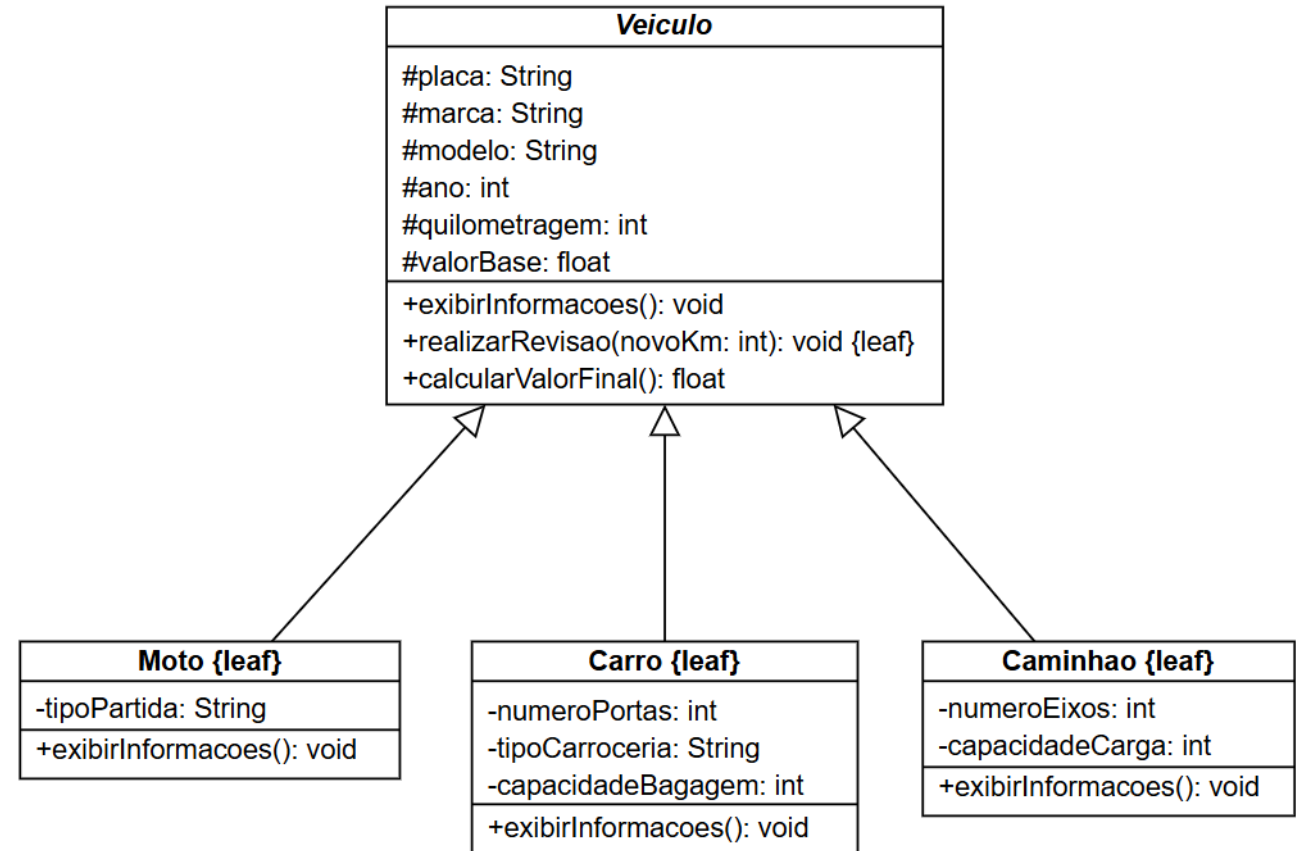
Não tem como o cálculo do perímetro ser diferente desse.



PRATICANDO

PRATICANDO

3. Aproveitando o projeto da Concessionaria, faça as seguintes modificações:
- Em Veículo:
 - Transforme a classe Veiculo em uma classe abstrata;
 - Indique que o método realizarRevisao não poderá ser sobrescrito.
 - Em todas as classes concretas (Moto, Carro e Caminhao):
 - Indique que a classe concreta não poderá ser herdada.
 - Sobrescreva o método exibirInformacoes para adicionar às informações bases do Veículo as específicas da classe concreta.



PRATICANDO

4. Escreva a hierarquia de classes e seus respectivos atributos e métodos para representar o seguinte cenário em um novo projeto Loja:
- Uma loja comercializa **3 tipos de mídia física: Vinil, BluRay e JogoDeVideogame**. Para **todos os produtos** existem: **nome, código, descrição, preço, peso** e **localização** (sigla do Estado em que o item se encontra).
 - **Vinil** é de **um ou mais artistas** e possui **apenas um gênero musical**.
Cada artista tem um **nome artístico**.
Os gêneros musicais são: “samba”, “rock”, “clássico”, “pop”, “forró”, “gospel” e “eletrônica”.
 - **BluRay** possui **um diretor** e **um ou mais atores** e possui **apenas um gênero cinematográfico**.
Cada diretor/ator tem um **nome artístico**.
Os gêneros cinematográficos são: “drama”, “comédia”, “ação”, “suspense”, “documentário”, “animação” e “ficção científica”.
 - **JogoDeVideogame** pertence a **uma plataforma**. As plataformas são: “PC”, “PlayStation”, “Xbox” e “Nintendo”.
Um jogo pode ter **um ou mais estúdios** (cada estúdio tem **nome**).
 - **Regras de frete → Método calcularFrete(ufDestino):**
 - Para entregar um produto, o **cálculo do frete** é feito multiplicando o **peso** do produto por **R\$ 3,50**.
A quantia de **R\$ 10,00** deve ser **acrescentada** ao valor do frete sempre que o **Estado de entrega** for **diferente** da **localização** do produto.
Itens **frágeis** (Vinil e BluRay) exigem **embalagem especial** de **R\$ 5,00**, que deve ser adicionada ao frete.
A **ausência** de qualquer informação envolvida no cálculo (peso, UF de origem, UF de destino) deve ser **impedir** a realização do cálculo.



INTERFACES

Indicando Contratos

RELACIONAMENTOS

- Os objetos interagem entre si através de relacionamentos, que são mecanismos pelos quais um objeto acessa recursos providos por outro.
- São tipos possíveis de relacionamentos entre classes:
 - Dependência
 - Associação
 - Agregação (todo-parte + fraca)
 - A parte existe sem o todo
 - O todo NÃO controla o ciclo de vida da parte
 - Composição (todo-parte + forte)
 - A parte NÃO existe sem o todo
 - O todo CONTROLA o ciclo de vida da parte
 - Generalização (Herança)
 - Implementação (Interface) → comprometer-se com um contrato

CONTRATO?

Imagine você entrando num restaurante para jantar. Você já sabe o que fazer para conseguir comer sua comida. O **menu** e o **garçom** formam a **interface** entre você (usuário) e a **cozinha** (implementação).

- Pelo menu/garçom você consegue:
 - **fazerPedido(prato, quantidade)**
 - **consultarTempoEspera()**
 - **alterarPedido(item, novaOpção)**
 - **cancelarPedido(item)**

O que é “contrato” aqui?

- **Nomes e opções estáveis** (o que é “Lasanha à Bolonhesa” não muda a cada minuto).
- **Regras claras:** *só dá pra pedir o que está disponível, cancelamentos só são possíveis até um certo momento, existe um tempo médio de preparo.*
- **Resultados esperados:** ao pedir, você recebe confirmação.

O que fica escondido (implementação)?

- Como a cozinha organiza as estações, a receita exata, as marcas de ingredientes, quem lava a louça...
- Mudar o fogão ou contratar outro chef **não deveria mudar a interface** com você (o menu e a maneira de pedir).

INTERFACES

- Uma interface especifica o “**contrato**” de funcionamento de uma ou mais classes.
- A interface, nesse caso, irá especificar **quais operações** as classes que a **implementam** devem possuir, mas não indica como essas operações devem ser implementadas.
- É comum confundir interface com classes abstratas que possuem somente métodos abstratos, porém, a herança pressupõe a existência de uma hierarquia conceitual, já a interface existe para modelar uma capacidade.
 - **Herança** (generalização/especialização) → modela hierarquia conceitual (kind-of / é-um).
 - Ex.: Ave → Pinguim. Pinguim é uma Ave.
 - **Interface** (contrato/implementação) → modela papel/capacidade (can-do / conforma ao contrato), não uma taxonomia.
 - Ex.: Nadador → Pinguim sabe Nadar, mas isso não o torna um “subtipo conceitual” de Nadador no sentido taxonômico; é apenas um papel que ele desempenha.
 - Além disso, podemos dizer que existem animais que não são Aves que sabem Nadar, mas não podemos dizer que toda Ave sabe Nadar.

INTERFACES

Taxonomia (herança): Forma → Retângulo, Círculo, Triângulo.

Retângulo é uma Forma. Círculo é uma Forma. Triângulo é uma Forma.

Papéis (interfaces): Desenhável, Pagável, Comparável.

Desenhável: Forma pode ser Desenhável, porém muito mais coisa pode também ser Desenhável, como uma Linha, um Botão, um ItemGráfico de um jogo, uma Imagem... Repara que eles podem não ter nada a ver conceitualmente um com os outros.

Pagável: Fatura, Boleto, Cartão implementam Pagável, embora não façam parte de uma mesma família conceitual.

Comparável: Qualquer coisa pode ser Comparável.

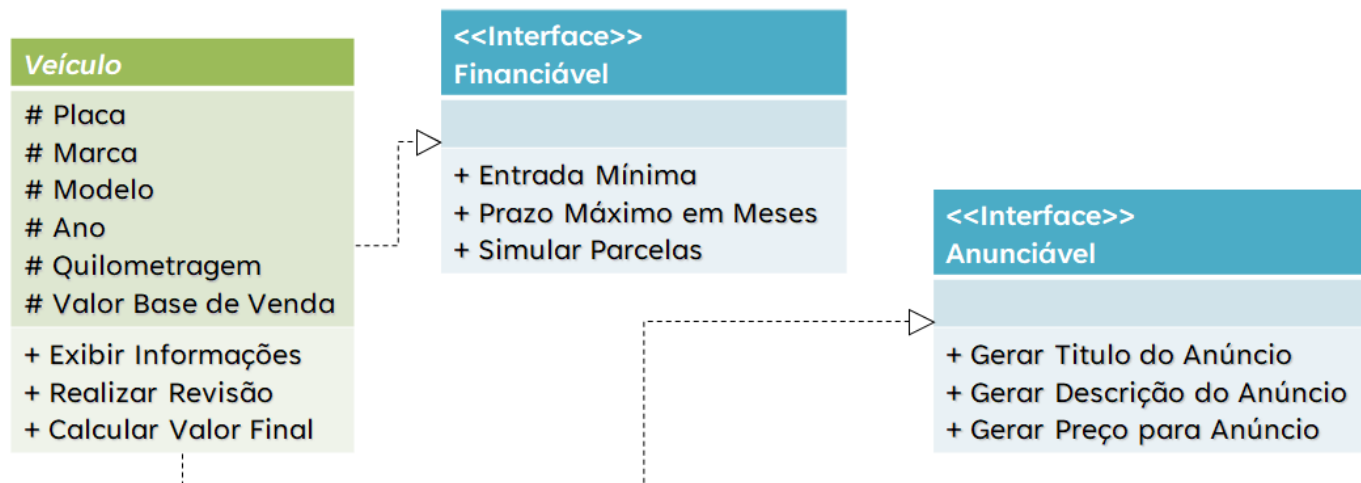
INTERFACES

O contrato de uma interface é os **métodos declarados** nessa interface.



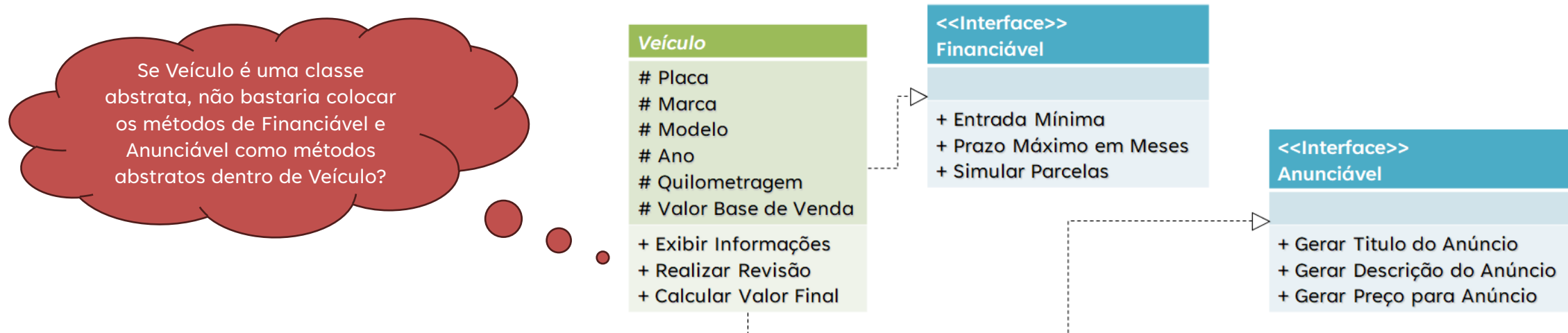
INTERFACES

- O relacionamento entre uma classe e uma interface é um relacionamento de **implementação**.
- Assim, a classe que **implementa** uma interface se compromete a implementar **todos os métodos** dessa interface, comprometendo-se a **cumprir o contrato**.
- Uma classe pode cumprir quantos contratos forem necessários:



INTERFACES

- O relacionamento entre uma classe e uma interface é um relacionamento de **implementação**.
- Assim, a classe que **implementa** uma interface se compromete a implementar **todos os métodos** dessa interface, comprometendo-se a **cumprir o contrato**.
- Uma classe pode cumprir quantos contratos forem necessários:



INTERFACES

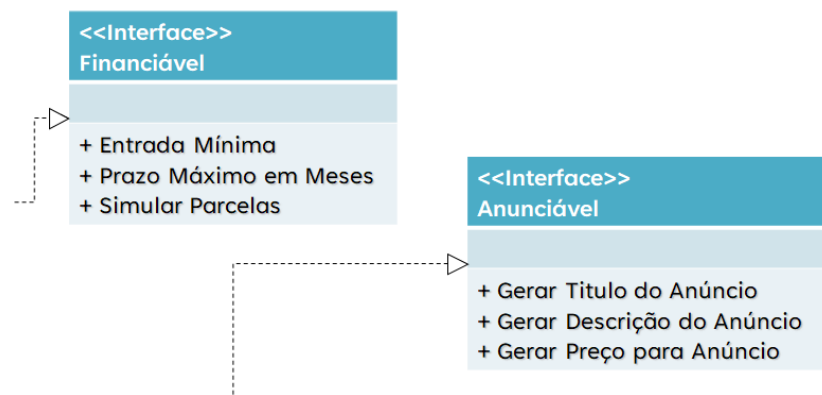
- Financiamento é um **papel/capacidade** (*can-do*), **não** parte da **essência** de um Veículo. Se amanhã a loja passar a financiar **acessórios, garantias estendidas** ou outros itens **que não são Veículos**, a lógica de financiamento teria que ser duplicada nessas outras classes ou ainda heranças forçadas teriam que ser criadas.
- Se existisse um Simulador de Financiamento que fizesse justamente o uso dos métodos de classes que implementam de Financiável, ele teria que depender não de Financiável, mas de Veículo, o que criaria uma **dependência desnecessária com Veículo** que poderia se estender no futuro para a dependência de outros produtos que pudessem ser vendidos e também financiados, **aumentando ainda mais o acoplamento**.
- Poderíamos ainda usar uma interface **para especificar papel/capacidade de somente alguns tipos de Veículos e não de todos, como Agendável** – *teste drive* – que deveria ser implementado somente por Motos e Carros.



EM JAVA

INTERFACES

- **Em Java**, a definição de uma interface é muito parecida com a de uma classe, só que ela somente define os métodos sem implementá-los!



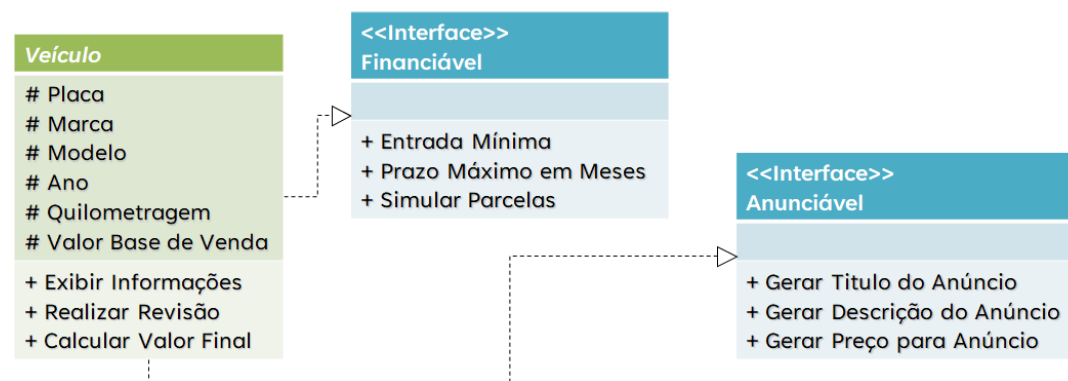
```
public interface Financiavel {
    public float entradaMinima();
    public int prazoMaximoEmMeses();
    public float simularParcelas(float entrada, int meses, float taxaAnual);
}
```

```
public interface Anunciavel {
    public String gerarTituloAnuncio();
    public String gerarDescricaoAnuncio();
    public float gerarPrecoParaAnuncio();
}
```

Assim como as classes, uma interface deve estar em um arquivo próprio, com o mesmo nome da interface.

INTERFACES

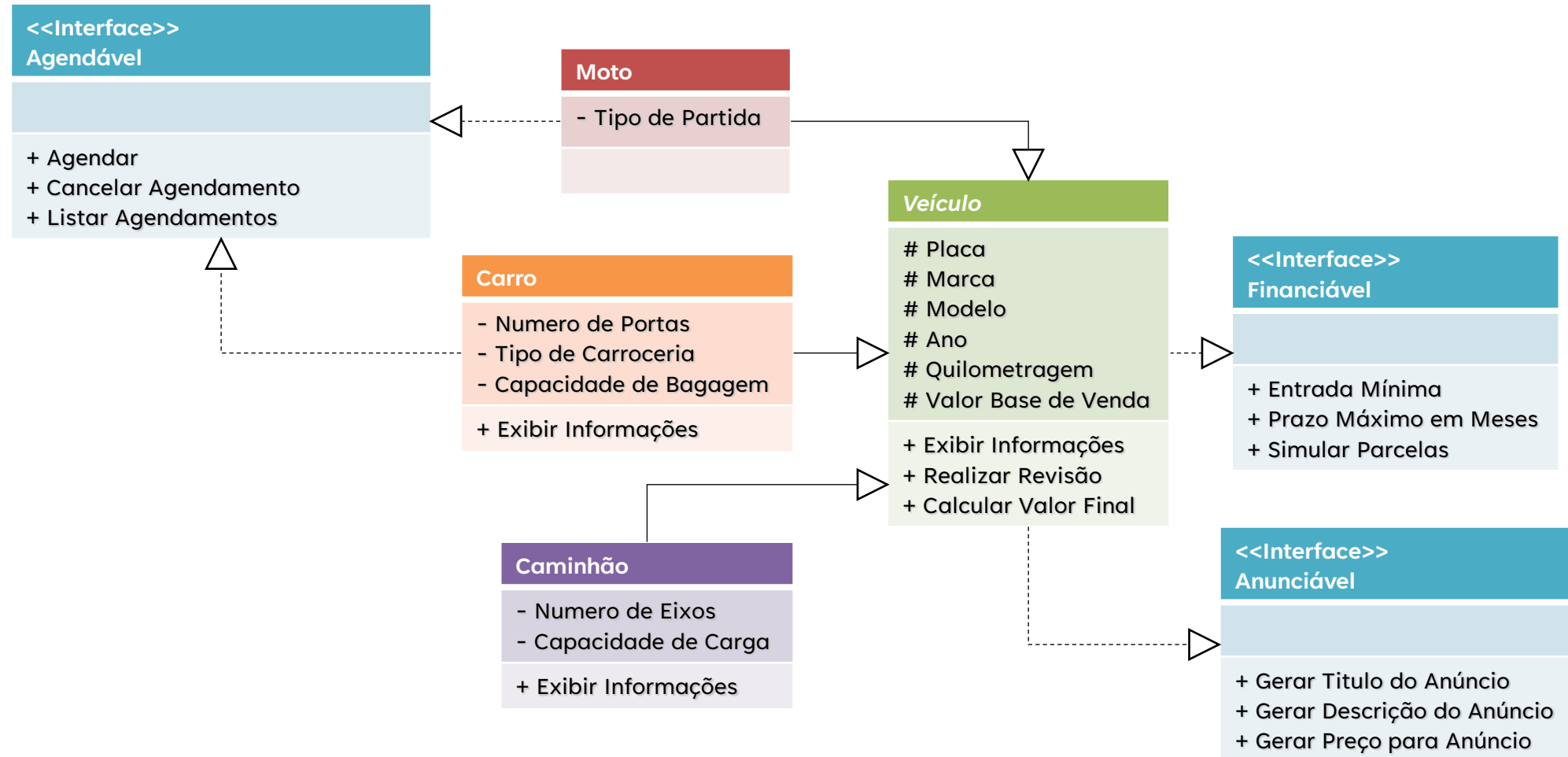
- Em Java, o relacionamento de implementação é definido pela palavra **implements**:



```
public abstract class Veiculo implements Financiavel, Anunciavel {
    ...
    public float entradaMinima() {...}
    public int prazoMaximoEmMeses() {...}
    public float simularParcelas(float entrada, int meses, float taxaAnual) {...}

    public String gerarTituloAnuncio() {...}
    public String gerarDescricaoAnuncio() {...}
    public float gerarPrecoParaAnuncio() {...}
}
```

EXTENSÃO E IMPLEMENTAÇÃO JUNTAS?



EXTENSÃO E IMPLEMENTAÇÃO JUNTAS?

```
public final class Moto extends Veiculo implements Agendavel {  
    // Sobrescreve o método herdado da superclasse  
    public void exibirInformacoes() { ... }  
  
    // Implementa os métodos da interface Agendavel  
    public boolean agendar(LocalDate data, Cliente cliente) { ... }  
    public boolean cancelarAgendamento() { ... }  
    public List<Agendamento> listarAgendamentos() { ... }  
  
    // Implementa os métodos da interface Financiavel  
    public float entradaMinima() { ... }  
    public int prazoMaximoEmMeses() { ... }  
    public float simularParcelas(float entrada, int meses, float taxaAnual) { ... }  
  
    // Implementa os métodos da interface Anunciavel  
    public String gerarTituloAnuncio() {...}  
    public String gerarDescricaoAnuncio() {...}  
    public float gerarPrecoParaAnuncio() {...}  
}
```

INTERFACES – ASPECTOS IMPORTANTES

- É possível definir atributos, variáveis ou parâmetros **do tipo da interface** (ATENÇÃO: NÃO DENTRO DA INTERFACE!)
- Porém, não é possível criar objetos usando uma interface, pois interfaces não são classes.

```
Financiavel f;
```

```
f = new Financiavel();
```

Erro de compilação!

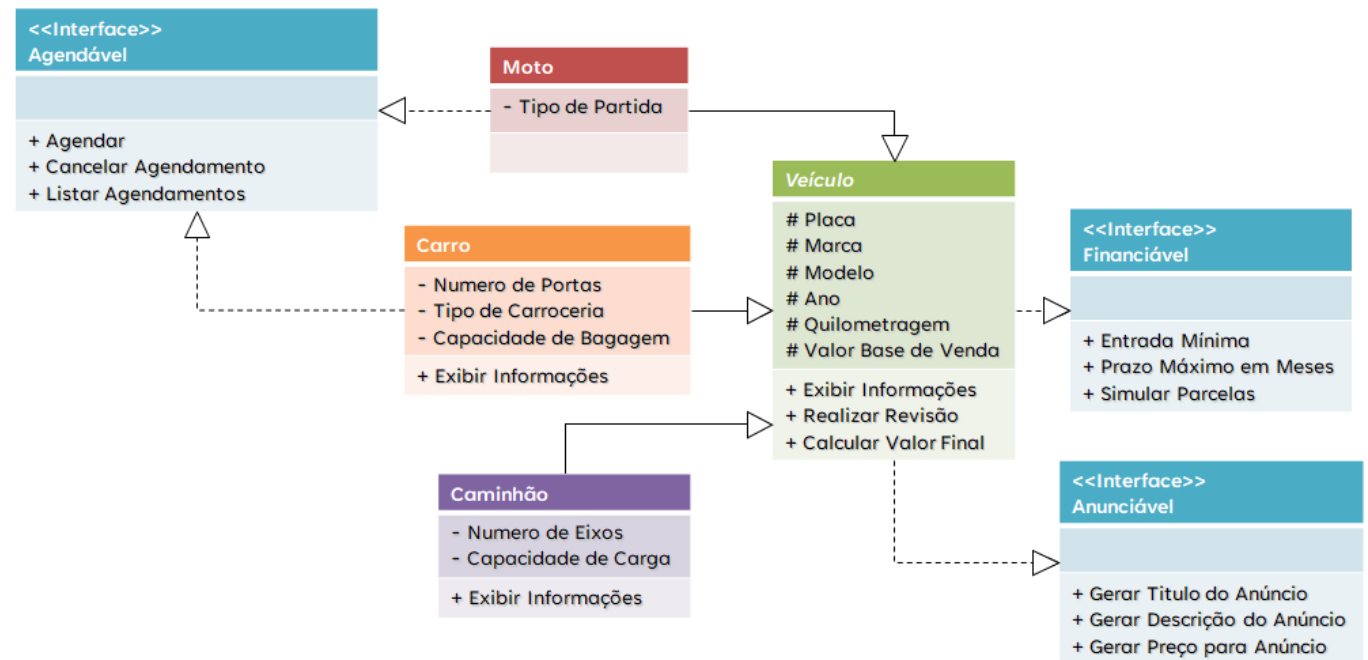
INTERFACES – ASPECTOS IMPORTANTES

- É possível ter variáveis do tipo da interface **referenciando** objetos de classes que **implementam** essa interface.

```
Financiavel f1, f2;  
Agendavel a;
```

```
f1 = new Carro();  
a = new Carro();
```

```
f2 = new Moto();
```



INTERFACES – ASPECTOS IMPORTANTES

- É possível chamar métodos usando variáveis do tipo da interface.
- Os **métodos chamados** devem ser **somente aqueles definidos na interface**.

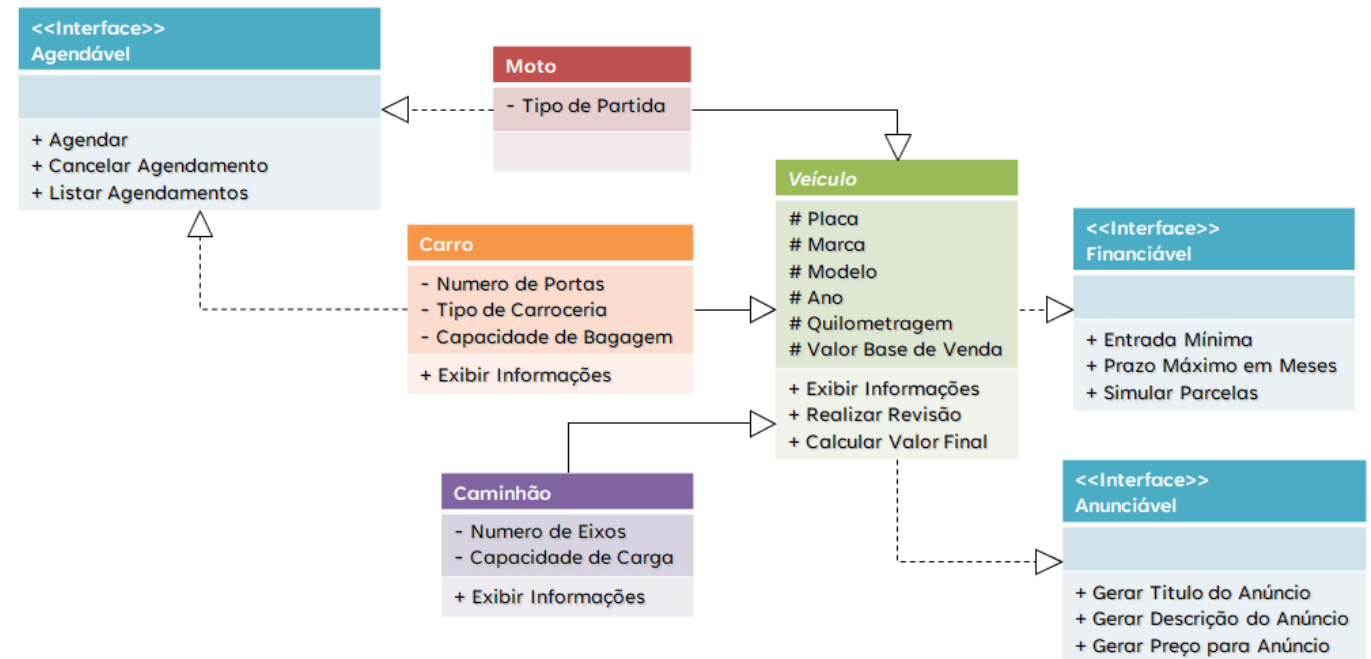
```
Financiavel f1, f2;  
Agendavel a;
```

```
f1 = new Carro();  
a = new Carro();
```

```
f2 = new Moto();
```

```
f1.simularParcelas();  
a.agendar();  
f1.realizarRevisao();  
a.exibirInformacoes();
```

```
f2.verificarEntradaMinima();  
f2.calcularValorFinal();
```



Erro de compilação!

INTERFACES – ASPECTOS IMPORTANTES

- É possível chamar métodos usando variáveis do tipo da interface.
- Os **métodos chamados** devem ser **somente aqueles definidos na interface**.

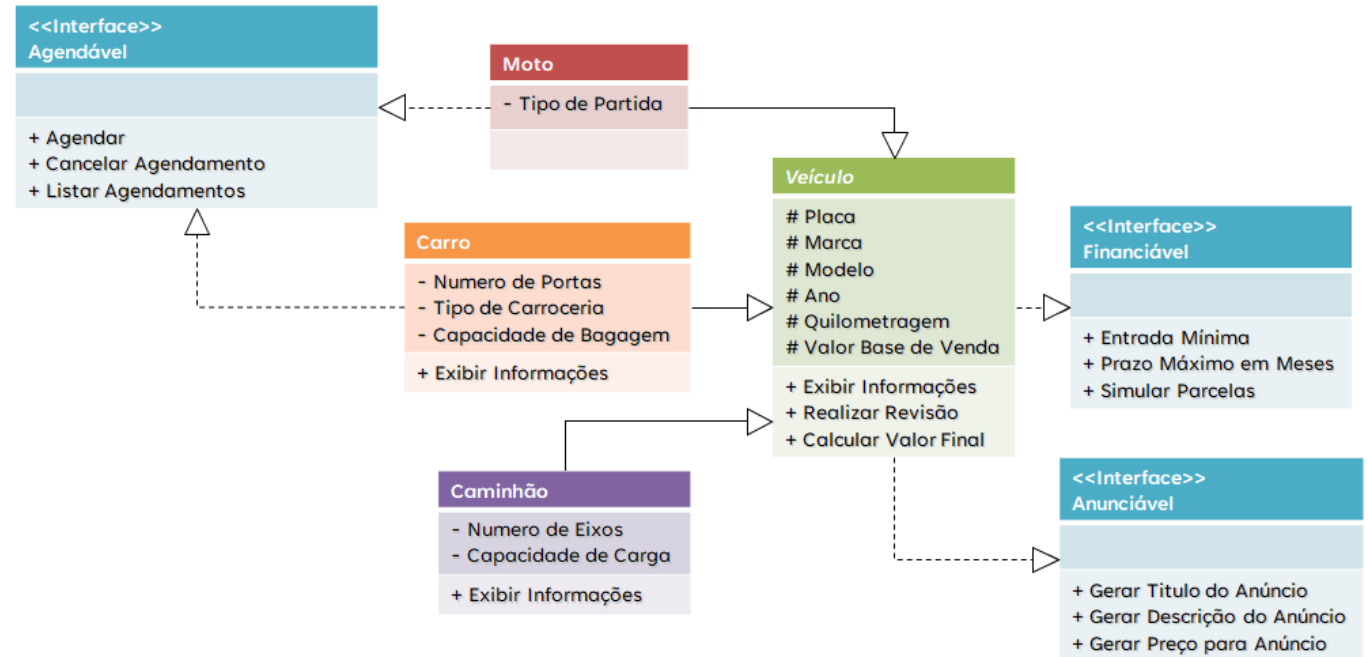
```
Financiavel f1, f2;  
Agendavel a;
```

```
f1 = new Carro();  
a = new Carro();
```

```
f2 = new Moto();
```

```
f1.simularParcelas();  
a.agendar();  
f1.realizarRevisao();  
a.exibirInformacoes();
```

```
f2.verificarEntradaMinima();  
f2.calcularValorFinal();
```



Erro de compilação!

INTERFACES – ASPECTOS IMPORTANTES

- É possível chamar métodos usando variáveis do tipo da interface.
- Os **métodos chamados** devem ser **somente aqueles definidos na interface**.

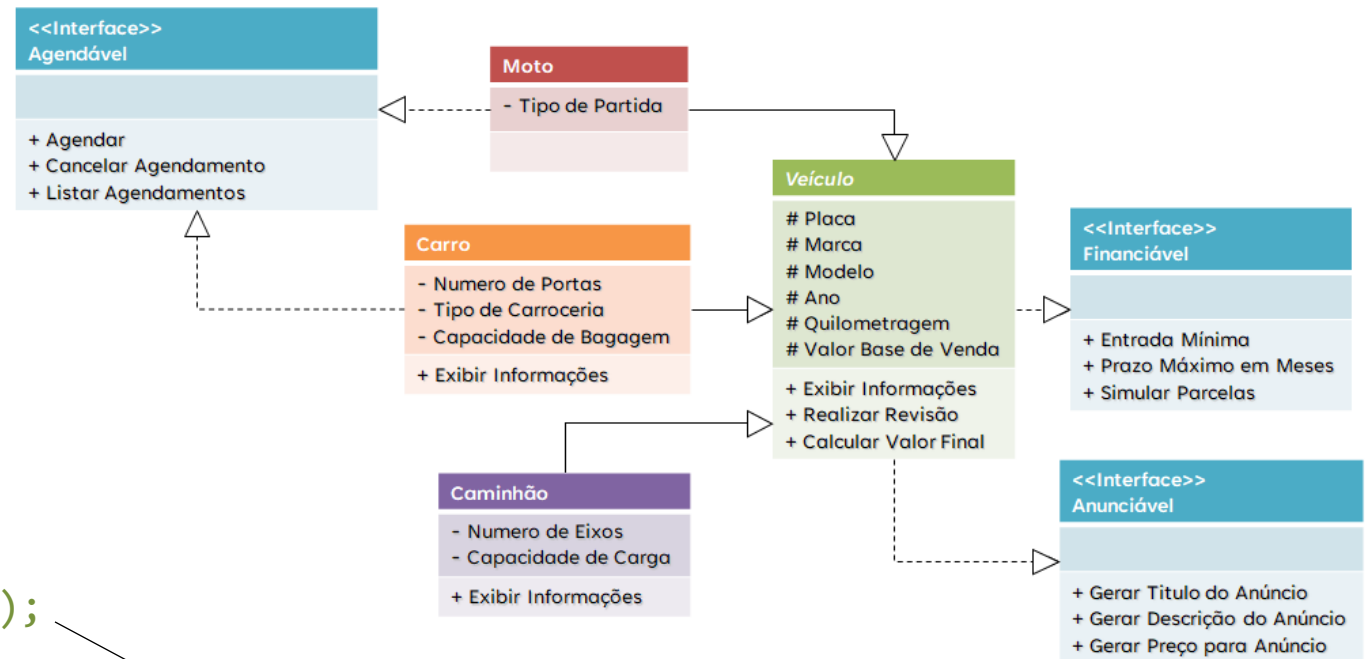
```
Financiavel f1, f2;  
Agendavel a;
```

```
f1 = new Carro();  
a = new Carro();
```

```
f2 = new Moto();
```

```
f1.simularParcelas();  
a.agendar();  
((Carro)f1).realizarRevisao();  
((Veiculo)a).exibirInformacoes();
```

```
f2.verificarEntradaMinima();  
((Moto)f2).calcularValorFinal();
```



Chama exibirInformacoes() de Veículo ou de Carro?

INTERFACES – ASPECTOS IMPORTANTES

- É possível chamar métodos usando variáveis do tipo da interface.
- Os **métodos chamados** devem ser **somente aqueles definidos na interface**.

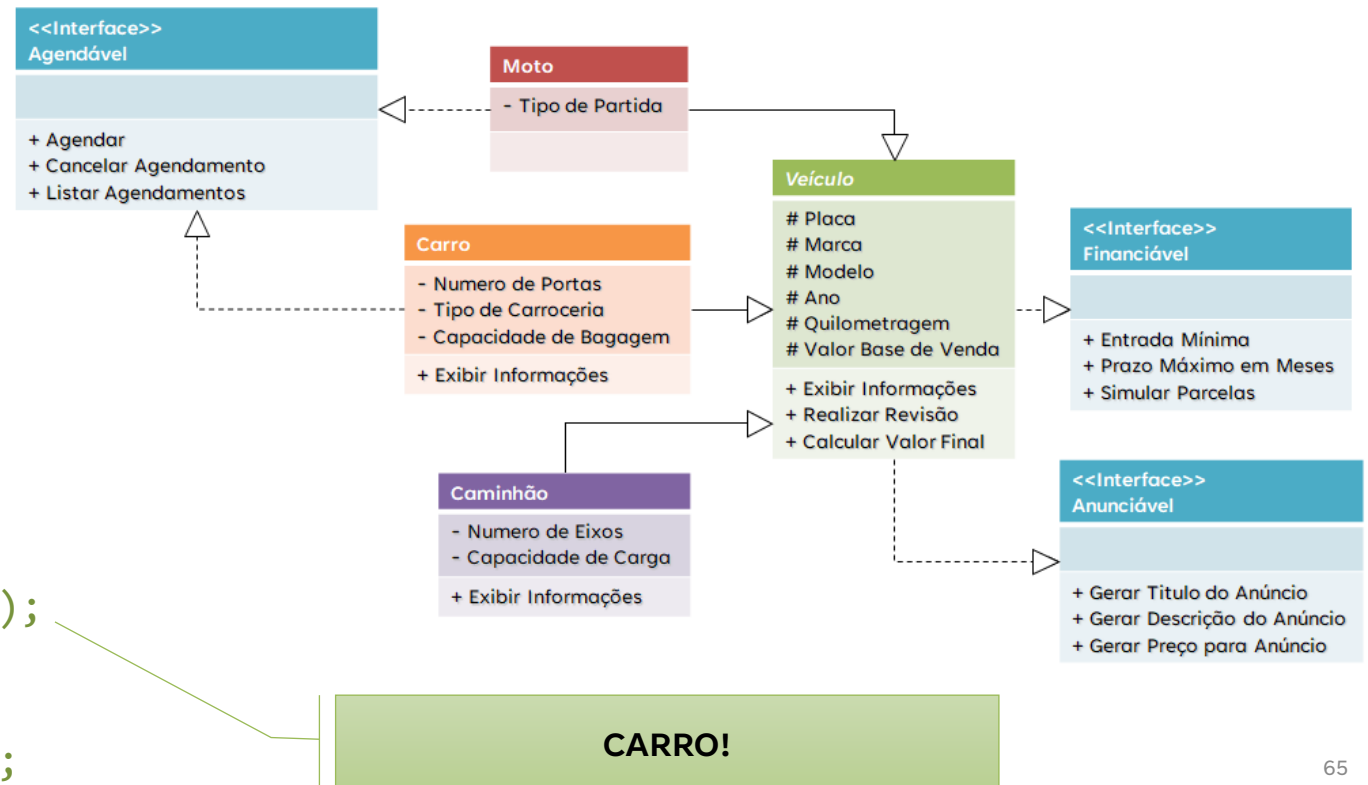
```
Financiavel f1, f2;  
Agendavel a;
```

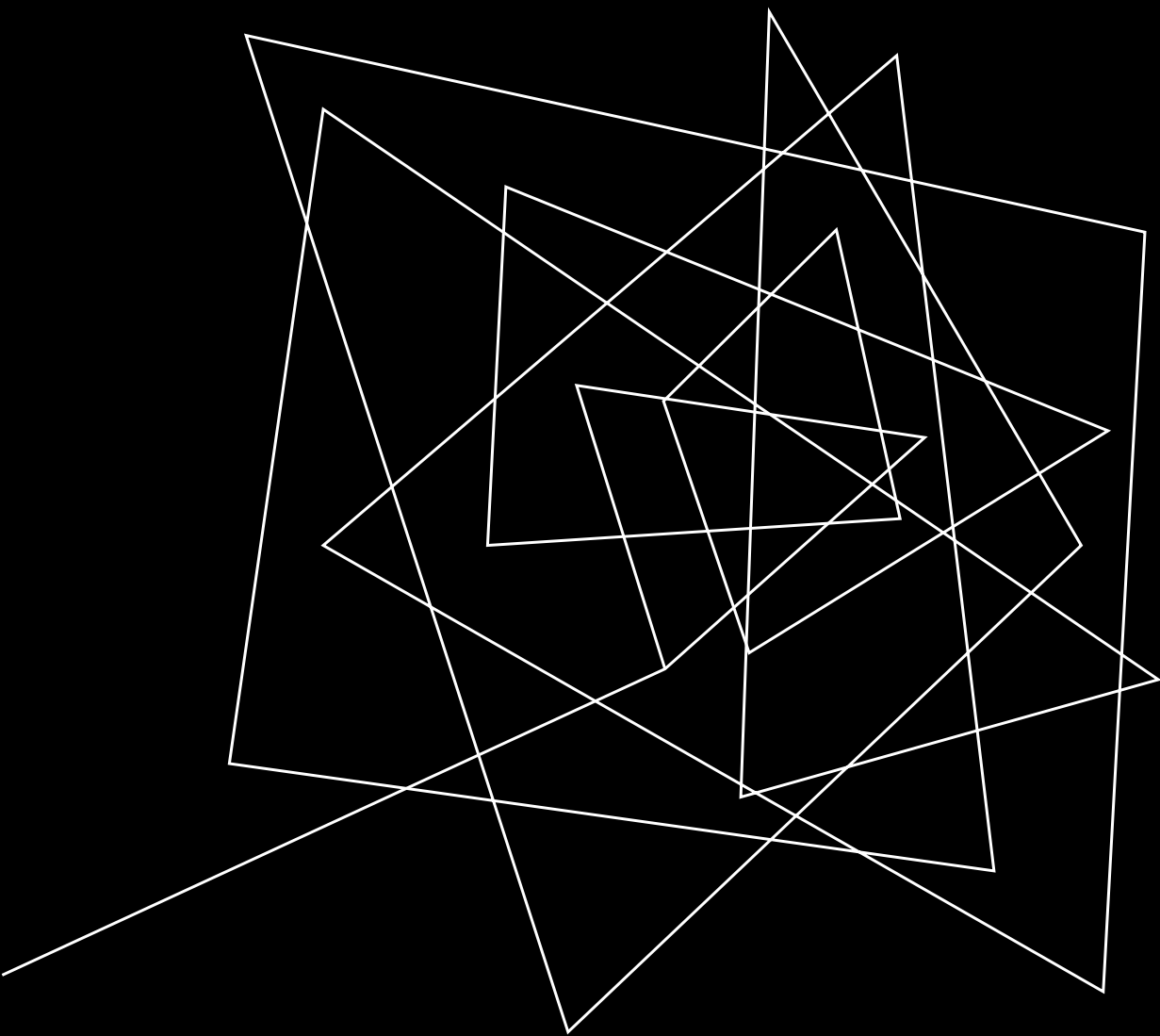
```
f1 = new Carro();  
a = new Carro();
```

```
f2 = new Moto();
```

```
f1.simularParcelas();  
a.agendar();  
((Carro)f1).realizarRevisao();  
((Veiculo)a).exibirInformacoes();
```

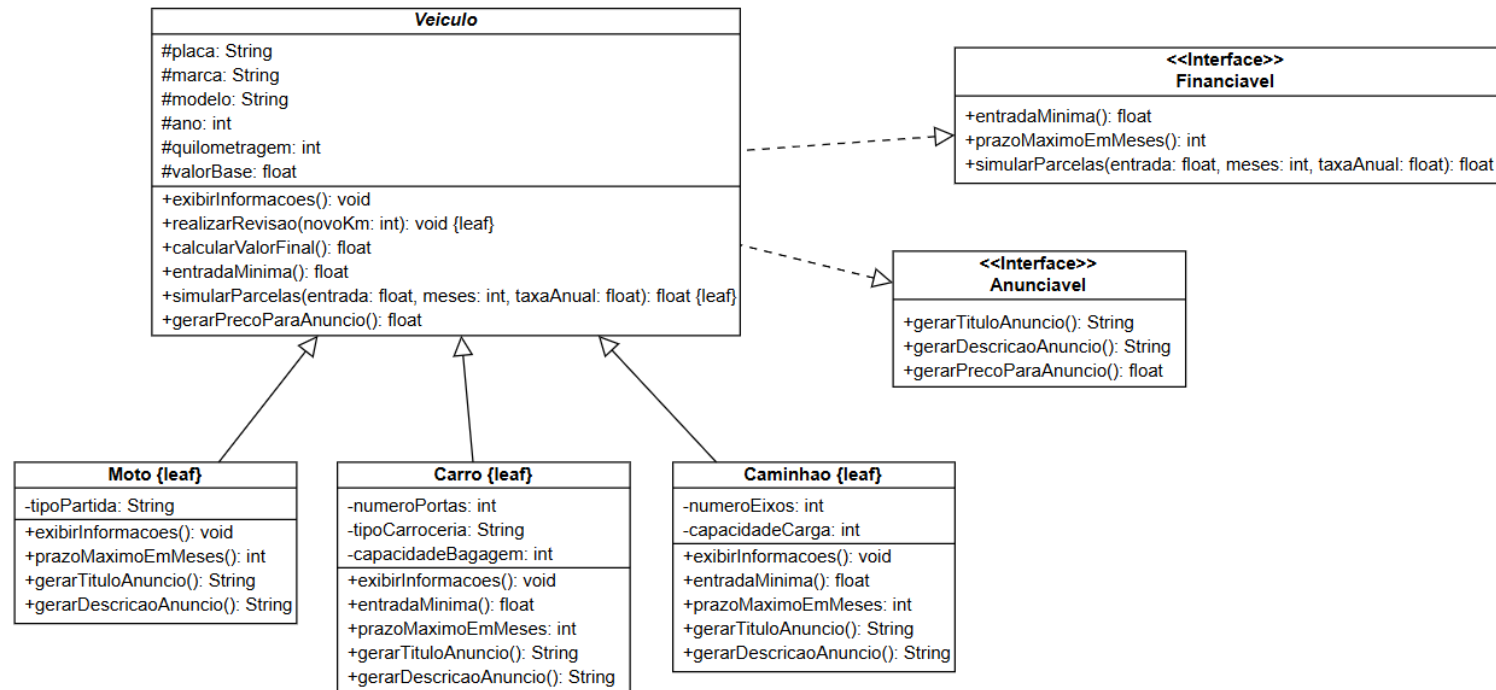
```
f2.verificarEntradaMinima();  
((Moto)f2).calcularValorFinal();
```





PRATICANDO

PRATICANDO



5. Aproveitando o projeto da Concessionaria, faça as seguintes modificações:

- Em Veículo:
 - Indique a implementação das interfaces **Financiavel** e **Anunciavel**
 - Implemente o método `entradaMinima()` que retorna 10% do valor base.
 - Implemente o método `simularParcelas()` que faz um cálculo simplificado da Tabela Price, indicando ele como final.
 - Implemente o método `gerarPrecoParaAnuncio()` que deve retornar o valor final do veículo com 10% de desconto.
- Em todas as classes concretas (**Moto**, **Carro** e **Caminhao**):
 - Implemente o outro método de **Financiavel**: `prazoMaximoEmMeses()`
 - para **Moto**, 30; para **Carro**, 60; para **Caminhao** 120.
 - Implemente os dois outros métodos de **Anunciavel**: `gerarTituloAnuncio()` e `gerarDescricaoAnuncio()`
- Sobrescreva o método `entradaMinima()` nas classes concretas **Carro** e **Caminhão**:
 - para **Carro**, 20% do valor base; e para **Caminhao**, 30%.

PRATICANDO

6. Aproveitando o projeto da Loja, implemente a interface `Calculavel`, que define o método `calcularTotal()`, cujo objetivo é retornar um valor numérico representando um total calculado.
 - Em seguida, refatore a classe abstrata de `Midia` para que ela implemente a interface `Calculavel` de forma que o método `calcularTotal()` retorne o preço da mídia + o valor do frete.
 - Crie uma classe `NotaFiscal` que armazena o identificador da nota, sua data e um array de `Calculavel`. Essa classe deve implementar a interface `Calculável`, de forma que o método `calcularTotal()` retorne o total da venda de todas as mídias da nota fiscal.
 - Teste o código criando 3 objetos do tipo `Midia`, sendo instanciados cada um com um tipo possível de `Midia`, bem como 1 objeto do tipo `NotaFiscal` com essas mídias dentro dela. Chame o método `calcularTotal()` da `NotaFiscal`.



CLASSE OBJECT DE JAVA

A Superclasse Raiz

A CLASSE OBJECT

- A classe Object em Java é a **superclasse raiz de todas as classes**. Isso significa que toda classe em Java herda direta ou indiretamente da classe `java.lang.Object` (importado automaticamente).
- A classe Object fornece métodos que são comuns a todos os objetos em Java já com suas implementações concretas, mas que podem ser sobrescritas por qualquer classe criada.

OS MÉTODOS DA CLASSE OBJECT

Método	Descrição
<code>equals(Object obj)</code>	Compara se dois objetos são "iguais". Por padrão, compara a referência na memória , mas pode ser sobrescrito para comparar conteúdo.
<code>hashCode()</code>	Retorna um número inteiro que representa o objeto. É usado em estruturas como HashMap e HashSet.
<code>toString()</code>	Retorna uma representação em string do objeto. Por padrão, retorna o nome da classe + código hash, mas é geralmente sobrescrito.
<code>getClass()</code>	Retorna a classe em tempo de execução do objeto (útil em reflexões).
<code>clone()</code>	Cria e retorna uma cópia do objeto atual (precisa implementar a interface Cloneable).
<code>finalize()</code>	Chamado pelo garbage collector antes de destruir um objeto (obsoleto e desaconselhado hoje em dia).
<code>notify()</code> , <code>notifyAll()</code> , <code>wait()</code>	Usados para comunicação entre threads , no controle de concorrência com synchronized.